Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет систем управления и робототехники

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

по дисциплине «Операционные системы»

«Настройка пользовательских демонов с применением systemd»

Выполнил: Очвада Адил Аса, 410561
Преподаватель: Шорохов Сергей Александрович

Санкт-Петербург
2026

1. Цель работы

Научиться настраивать пользовательские демоны с помощью системного менеджера systemd: написать bash-скрипты, оформить их в виде юнит-файлов `.service` и `.timer`, запустить демона по расписанию, управлять им с помощью `systemctl` и включить его в автозагрузку.

2. Теоретическая часть

Демон (daemon) – программа, которая работает в фоне и не взаимодействует с пользователем напрямую. Обычно демоны запускаются при старте системы, по таймеру или по событию. В современных дистрибутивах Linux запуском и работой демонов управляет системный менеджер systemd.

Для настройки демонов systemd использует юнит-файлы (юниты) – обычные текстовые файлы, каждый из которых описывает отдельный элемент системы. Юниты располагаются в трёх каталогах:

- `/etc/systemd/system` – пользовательские юниты;
- `/run/systemd/system` – юниты, создаваемые «на лету» во время работы системы;
- `/usr/lib/systemd/system` – юниты из установленных пакетов.

Тип юнита определяется расширением файла: `.service` – запуск конкретного демона; `.target` – группировка юнитов и синхронизация их запуска; `.timer` – запуск по расписанию; `.path` – запуск по событию в файловой системе; `.device` – запуск по событию подключения устройства.

С systemd работают из терминала утилиты `systemctl` (запуск, остановка, статус, автозагрузка), `journalctl` (журнал юнитов), `systemd-analyze` (время загрузки) и другие.

Юнит-файл состоит из секций. В секции `[Unit]` задаются описание (`Description`) и зависимости: `Wants` – юниты, запуск которых желателен, `After` – юниты, после запуска которых стартует данный. В секции `[Service]` указывается, что и как запускать: `Type` (`simple` – процесс запускается сразу и не ветвится), `Environment` (переменные окружения), `User` и `Group`, `WorkingDirectory`, `ExecStart` (команда запуска). Секция `[Install]` определяет, в каком режиме системы юнит включается в

автозагрузку (`WantedBy`).

Для вывода уведомлений на рабочий стол используется утилита `notify-send`: ключ `-i` задаёт значок уведомления, ключ `-t` – время показа в миллисекундах; далее в кавычках идут заголовок и дополнительный текст.

3. Экспериментальная часть

3.1. Подготовка и запись терминала

В виртуальной машине (Ubuntu MATE) открыт терминал и запущена запись команд `script -a record_result_lab_5`. Система ответила `Script started, output log file is 'record_result_lab_5'`. Затем выполнен переход на рабочий стол (`cd ~/Desktop`) и распакован архив с изображениями командой `tar -xvzf faces.tar.gz` (рисунок 1).

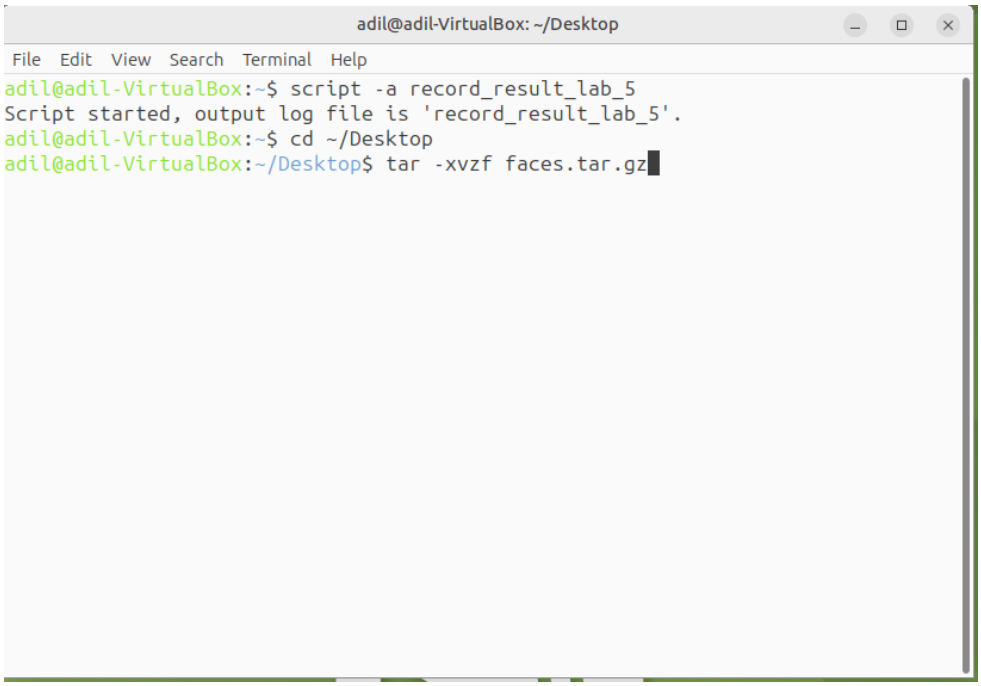


Рисунок 1 – Запуск записи терминала и распаковка архива

3.2. Генерация варианта

После распаковки командой `ls -l` проверено наличие файла `lab5.py` и каталога `images`. Вариант получен командой `python3 lab5.py` (рисунок 2):

– время отдыха (relax_time): 16;

– период повторения (repetition_period): 36 секунд;

– значок: `12235_2005.jpg`.

Скрипт `lab5.py` выводит время отдыха в минутах, но в методическом указании это значение передаётся скрипту как число секунд, поэтому в работе использовано значение 16 секунд.

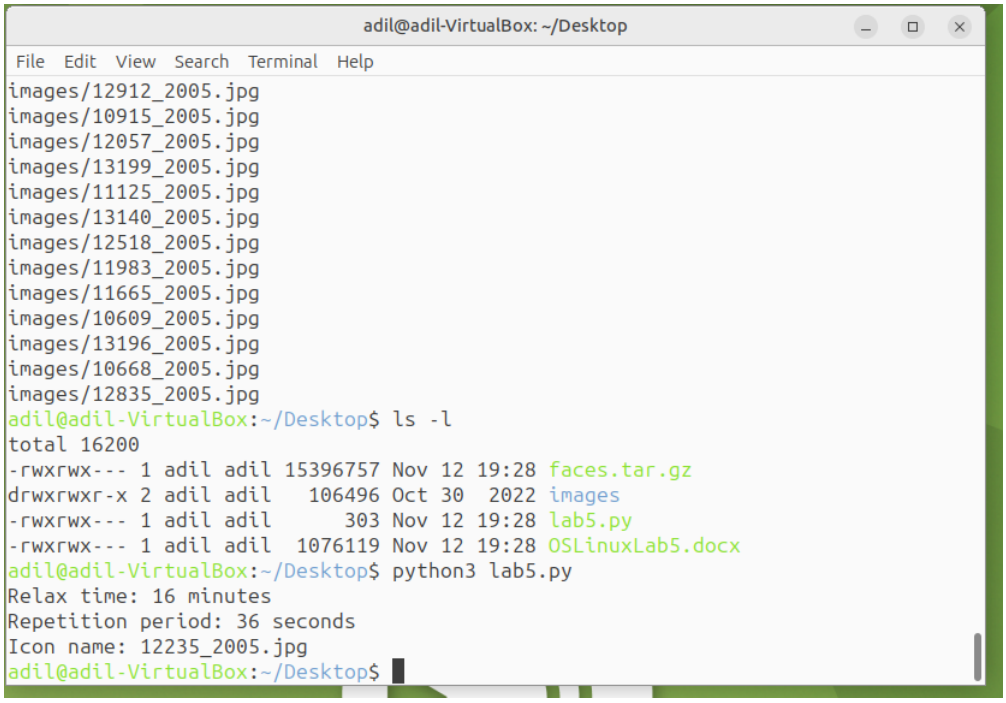


Рисунок 2 – Результат генерации варианта

3.3. Создание bash-скриптов

Для настройки демона нужны имя пользователя, путь к файлу авторизации X-сервера и адрес шины D-Bus. Они получены командами `whoami`, `xauth` и `echo $DBUS_SESSION_BUS_ADDRESS` (рисунок 3): пользователь `adil`, файл `/home/adil/.Xauthority`, адрес `unix:path=/run/user/1000/bus`. Команда `xauth` запускает собственную оболочку с приглашением `xauth>`, поэтому команда `echo` была выполнена после выхода из неё командой `exit`.

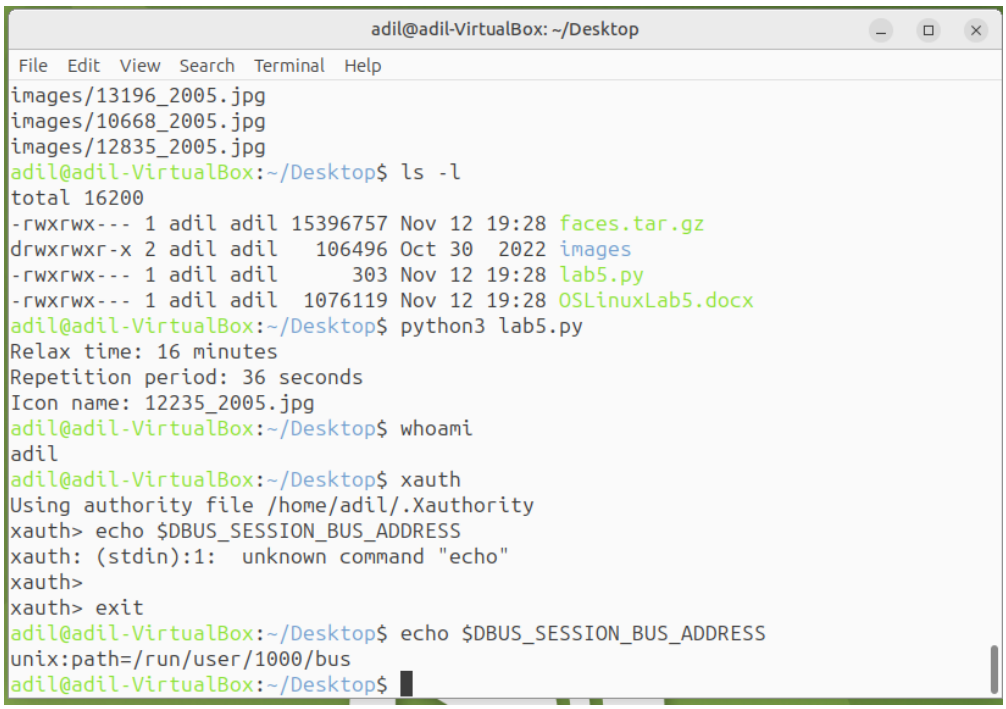


Рисунок 3 – Получение имени пользователя, пути `.Xauthority` и адреса D-Bus

На рабочем столе созданы два скрипта и сделаны исполняемыми (`chmod +x`, рисунок 4). Первый скрипт `eyes-start.sh` показывает уведомление о начале отдыха:

```
#!/bin/bash
period_s=$1
period_ms=$((period_s*1000))
icon=/home/adil/Desktop/images/12235_2005.jpg
notify-send -i $icon -t $period_ms "Time to relax, senpai!" "$period_s
   seconds"
```

Строка `#!/bin/bash` (шебанг) задаёт интерпретатор. Переменной `period_s` присваивается первый параметр скрипта, `period_ms` получается переводом секунд в миллисекунды, в переменной `icon` хранится абсолютный путь к значку из варианта. Команда `notify-send` показывает уведомление на время `period_ms`.

Второй скрипт `eyes-stop.sh` ждёт время отдыха, а затем выводит уведомление об окончании отдыха на 5 секунд:

```
#!/bin/bash
sleep 16
icon=/home/adil/Desktop/images/12235_2005.jpg
notify-send -i $icon -t 5000 "Time to get back to work, senpai!"
```

Работа скрипта проверена вручную командой `./eyes-start.sh 16`: на рабочем столе появилось уведомление со значком (рисунок 5). Ошибок в консоль не выведено.

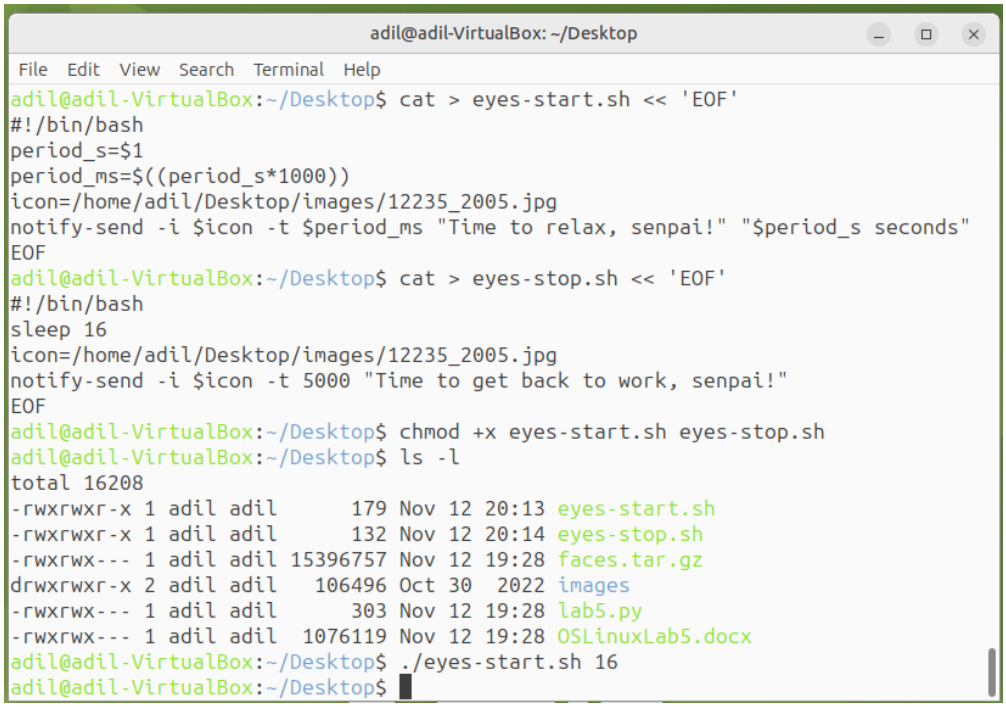


Рисунок 4 – Создание скриптов и проверка прав на выполнение

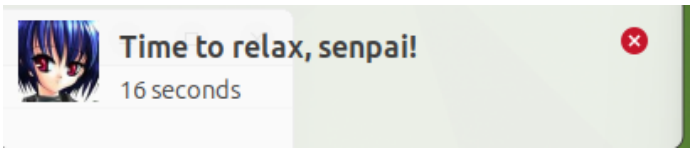


Рисунок 5 – Всплывающее уведомление

3.4. Создание юнит-файлов

В каталоге `/etc/systemd/system` с помощью `sudo tee` созданы три юнит-файла, затем им выданы права на выполнение (`sudo chmod +x`).

Юнит `eyes-relax.service` запускает скрипт `eyes-start.sh` с параметром 16 (рисунок 6):

```
[Unit]
Description=eye relax daemon
After=eyes-stop.service
Wants=eyes-stop.service

[Service]
Type=simple
Environment="DISPLAY=:0"
Environment="XAUTHORITY=/home/adil/.Xauthority"
Environment="DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus"
User=adil
Group=adil
WorkingDirectory=/home/adil/Desktop/
ExecStart=/home/adil/Desktop/eyes-start.sh 16
```

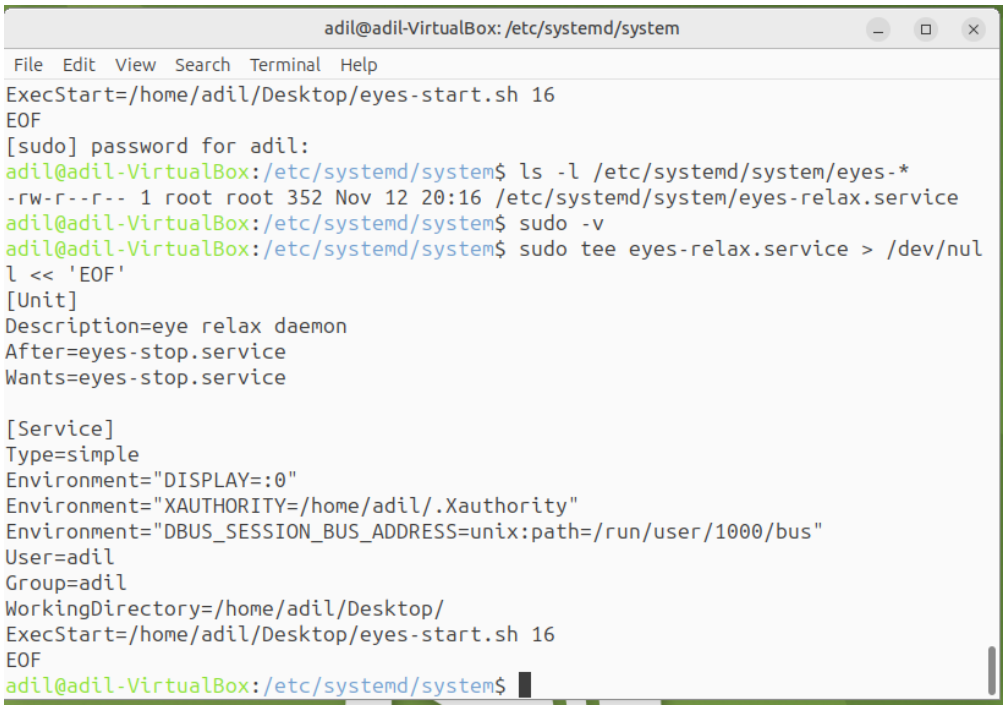


Рисунок 6 – Создание файла `eyes-relax.service`

Таймер `eyes-relax.timer` запускает одноимённую службу по расписанию (рисунок 7):

```
[Unit]
Description=scheduler for eyes service

[Timer]
OnBootSec=60s
OnUnitActiveSec=36s

[Install]
WantedBy=timers.target graphical.target
```

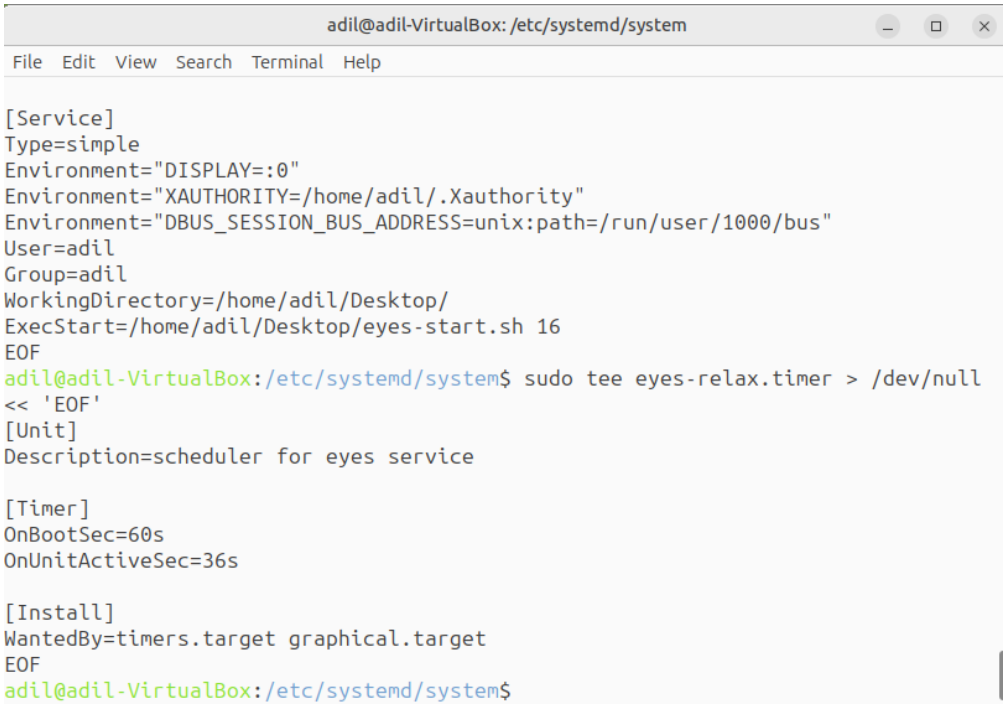


Рисунок 7 – Создание файла `eyes-relax.timer`

Юнит `eyes-stop.service` запускает скрипт `eyes-stop.sh` (рисунок 8):

```
[Unit]
Description=eye stop relax daemon

[Service]
Type=simple
Environment="DISPLAY=:0"
Environment="XAUTHORITY=/home/adil/.Xauthority"
Environment="DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus"
User=adil
Group=adil
WorkingDirectory=/home/adil/Desktop/
ExecStart=/home/adil/Desktop/eyes-stop.sh

[Install]
WantedBy=graphical.target
```

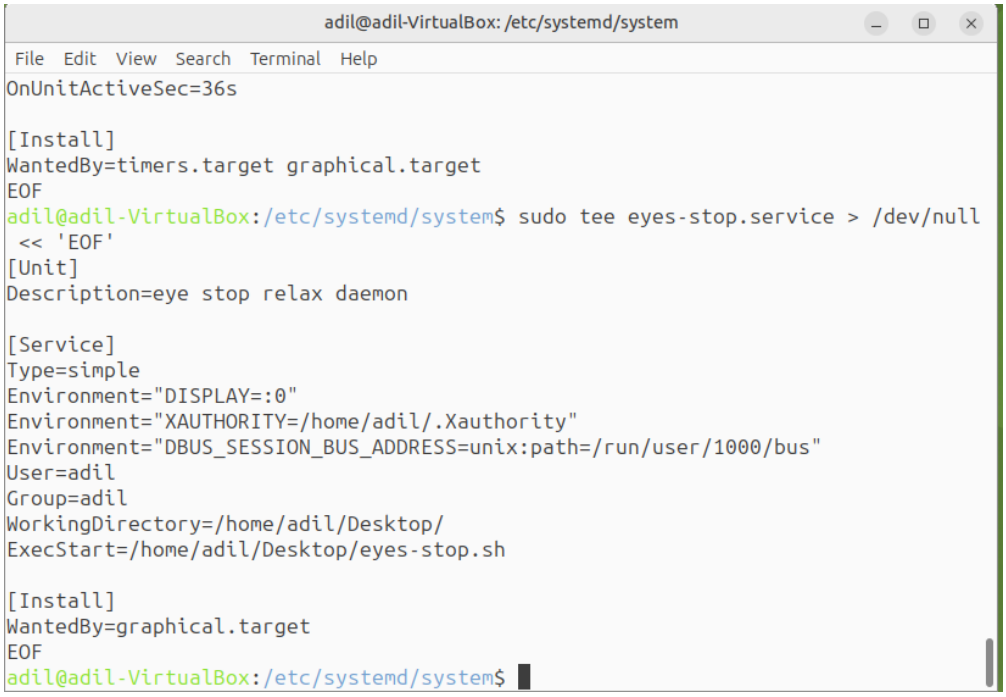


Рисунок 8 – Создание файла `eyes-stop.service`

В методическом указании в этот юнит добавлена строка `Requires=eyes-start.service`, но юнита с таким именем в работе нет (скрипт запускается службой `eyes-relax.service`), поэтому systemd не смог бы найти зависимость. Эта строка не использована. Список созданных файлов показан на рисунке 9.

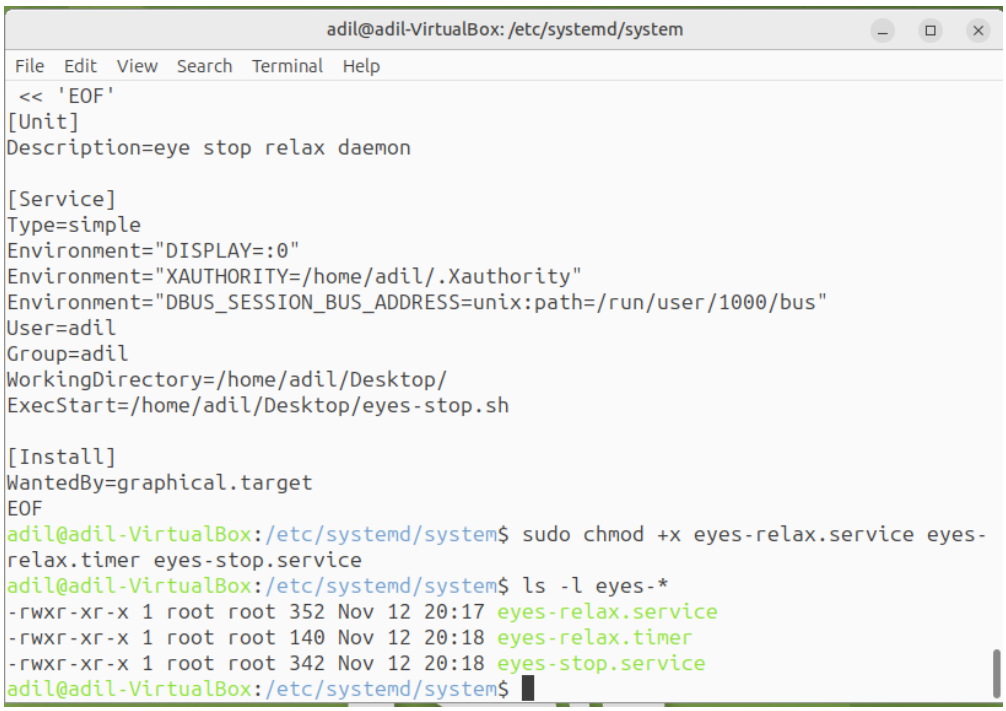


Рисунок 9 – Права на юнит-файлы

Секции и поля файла таймера.

– `[Unit]` – общая информация о юните; поле `Description` – текстовое описание, которое показывает `systemctl status`.

– `[Timer]` – расписание запуска. `OnBootSec=60s` – запуск через 60 секунд после загрузки системы; `OnUnitActiveSec=36s` – повторный запуск через 36 секунд после предыдущей активации службы (это период повторения из варианта).

– `[Install]` – режим включения в автозагрузку; поле `WantedBy=timers.target graphical.target` подключает таймер к целям таймеров и графического режима.

Как работает цепочка. Таймер каждые 36 секунд запускает службу `eyes-relax.service`. Она показывает уведомление о начале отдыха и через `Wants=eyes-stop.service` запускает вторую службу. Та ждёт 16 секунд (`sleep 16`) и показывает уведомление об окончании отдыха.

3.5. Запуск демона

Конфигурация загружена командой `sudo systemctl daemon-reload`, затем запущен таймер `sudo systemctl start eyes-relax.timer`. Статус таймера: `active (waiting)`, в поле `Triggers` указана служба `eyes-relax.service` (рисунок 10).

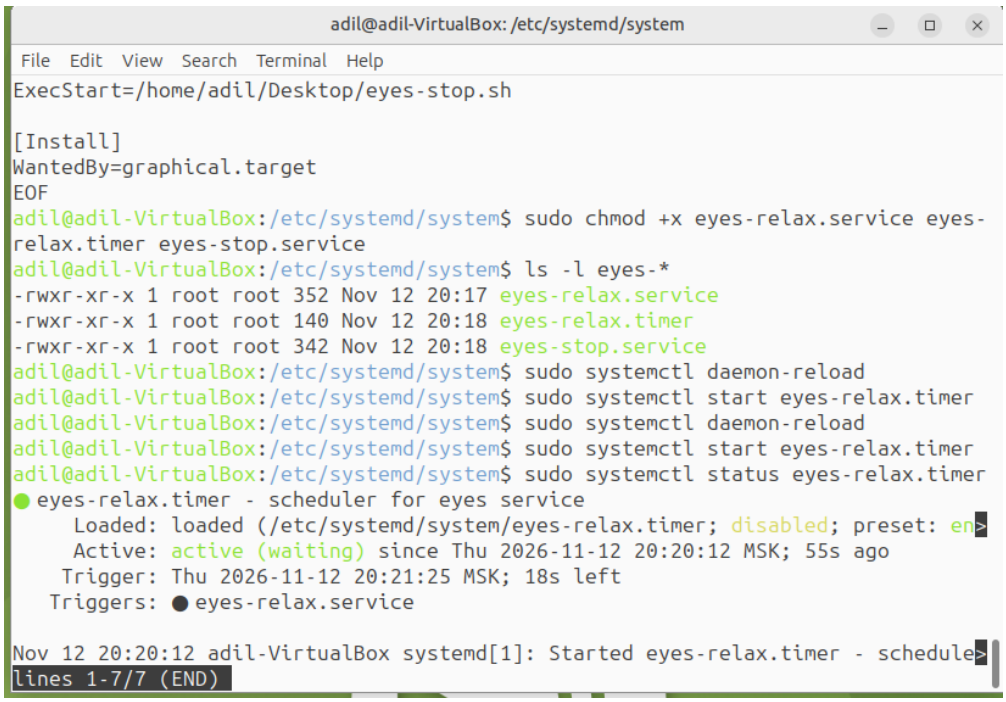


Рисунок 10 – Статус таймера `eyes-relax.timer` после запуска

Демон работает: на рабочем столе по очереди появляются уведомления о начале и об окончании отдыха (рисунок 11).

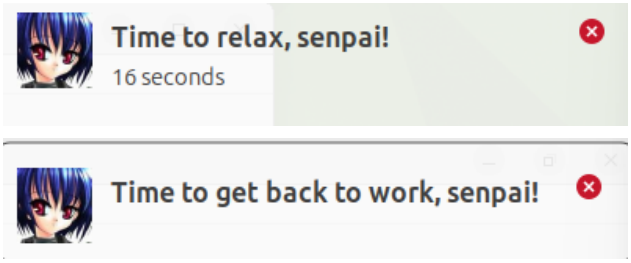


Рисунок 11 – Уведомления демона: начало и окончание отдыха

Статусы служб и автозагрузка проверены командами `systemctl status` и `sudo systemctl enable eyes-relax.timer` (рисунок 12):

– `eyes-relax.service` запущена таймером (`TriggeredBy: eyes-relax.timer`), отработала за 247 мс и завершилась с кодом `status=0/SUCCESS`;

– `eyes-stop.service` отработала 16,064 с (время `sleep 16`) и также завершилась с кодом `status=0/SUCCESS`;

– обе службы находятся в состоянии `inactive (dead)`: это нормально для коротких скриптов, которые завершаются после выполнения;

– команда `enable` создала символические ссылки в каталогах `timers.target.wants` и `graphical.target.wants`.



Рисунок 12 – Статусы служб и таймера, включение автозагрузки, завершение записи

Запись терминала остановлена командой `exit` (`Script done.`). Файл `record_result_lab_5` сохранён в домашнем каталоге.

3.6. Проверка после перезагрузки

Виртуальная машина перезагружена. В новом терминале команда `sudo systemctl status eyes-relax.timer --no-pager` показала `enabled` и `active (waiting)`: таймер запустился сам, без ручного запуска (рисунок 13).

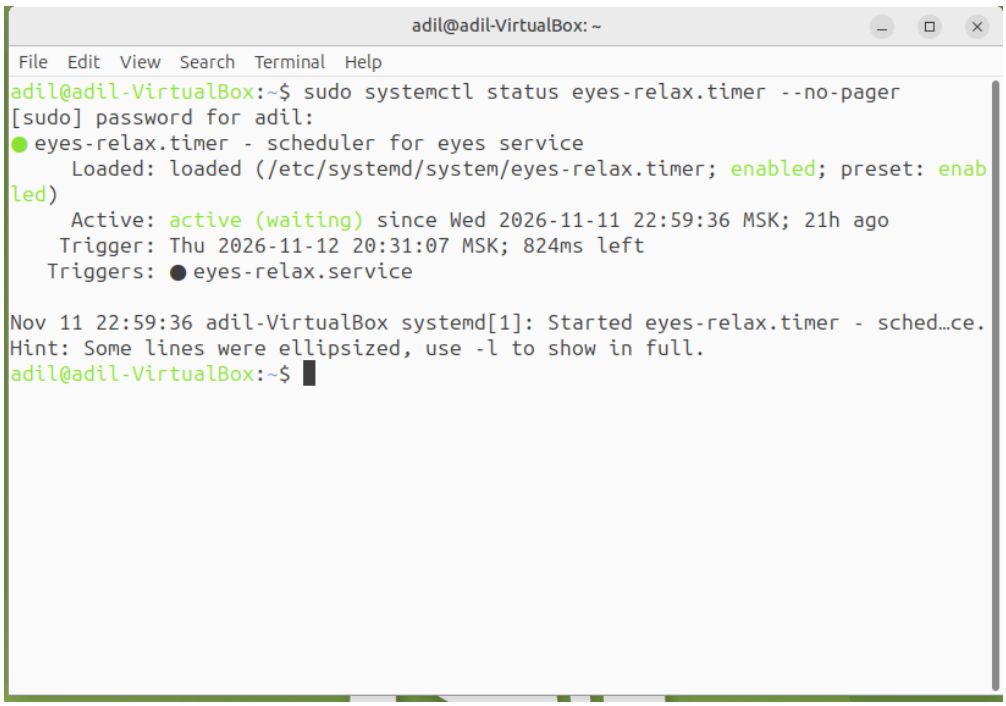


Рисунок 13 – Статус таймера после перезагрузки виртуальной машины

4. Трудности при выполнении

– Команда `xauth` открывает свою оболочку: команда `echo` после неё была воспринята как неизвестная команда `xauth`. Нужно было выйти командой `exit` и повторить `echo $DBUS_SESSION_BUS_ADDRESS`.

– При вставке нескольких команд сразу `sudo` запросил пароль посреди вставки, и часть строк не выполнилась. Помогли проверка `ls -l` и кэширование пароля командой `sudo -v`, после чего юнит-файлы были созданы заново.

– `systemctl status` открывает вывод в просмотрщике (pager), и следующие команды попадали в него, а не в оболочку. Выход – клавиша `q` и настройка `export SYSTEMD_PAGER=`.

– Строка `Requires=eyes-start.service` из методического указания ссылается на несуществующий юнит, поэтому она не использована.

– Время отдыха в `lab5.py` выводится в минутах, а в методическом указании используется как секунды. Использовано значение 16 секунд.

5. Заключение

В ходе работы получен вариант (время отдыха 16, период 36 с, значок `12235_2005.jpg`), написаны два bash-скрипта и три юнит-файла systemd: две службы и таймер. Демон показывает уведомление о начале отдыха каждые 36 секунд и через 16 секунд уведомление об окончании. Проверены статусы служб, включена автозагрузка таймера и подтверждено, что после перезагрузки системы таймер запускается автоматически. Освоены команды `systemctl` (`daemon-reload`, `start`, `status`, `enable`) и структура юнит-файлов. Цель работы достигнута.

Приложения

Файл `record_result_lab_5` – запись команд терминала; файлы `eyes-start.sh` и `eyes-stop.sh` – bash-скрипты; файлы `eyes-relax.service`, `eyes-relax.timer` и `eyes-stop.service` – юнит-файлы systemd.